



myP ROS Connection Manual

Version 1.4.4
(corresponds to the myP version)
2020-03-10

The information contained in this document is property of F&P Robotics AG and shall not be reproduced in whole or in part without prior written approval of F&P Robotics AG. The information provided herein is subject to change without notice and should not be constructed as a commitment by F&P Robotics AG. This manual is periodically reviewed and revised. F&P Robotics AG assumes no responsibility for any errors or omissions in this document.

This manual consists of original instructions from F&P Robotics AG and is intended for all users of myP. Before using the functions mentioned in this document together with myP and a P-Rob robotic arm or P-Grip robotic gripper, you must read the corresponding user manuals carefully, particularly paying attention to the safety instructions on the P- Rob user manual. Compliance with the instructions in all manuals is mandatory.

Copyright © 2011-2020 by F&P Robotics AG. All rights reserved.

The F&P logo as well as P-Rob®, P-Grip® and myP® are registered trademarks of F&P Robotics AG.

Contents

Prerequisites	4
Install ROS and Build Messages	4
Source ROS Environment	4
Communicating with the ROS Module	5
Reading Data from Topics	5
Calling myP Functions via Services	6
core/connect	6
core/calibrate	7
core/get_status	7
core/change_project	8
retrieving information through services	8
movement commands	9
core/disconnect	9
Future Changes	10

1 Prerequisites

This chapter contains instructions how to access myP running on a P-Rob via a ROS connection. Depending on your setup, you should only need to go through them once.

1.1 Install ROS and Build Messages

The Robot Operating System (ROS) is a framework for distributed computing, specifically created for robotic applications. It has been designed to run on Linux Ubuntu operating systems. Begin by choosing the ROS version that fits your Ubuntu distribution and follow the [ROS installation instructions](#). Once ROS is installed on your system, [create a catkin workspace](#). To communicate with myP through ROS, your workspace needs to contain a set of [ROS messages](#). You can clone them into the `src` directory of your workspace from our GitHub page. Here's a list of available ROS messages, sorted by use-case:

- [fp_core_msgs](#) (necessary core features to control P-Rob via ROS)
- [fp_pgrip_msgs](#) (if you want to control a P-Grip via ROS in addition)
- [fp_learning_msgs](#) (if you are using the myP Learning Module in addition)

You will definitely require the **fp_core_msgs** on your system, and perhaps additional ones, depending on the add-ons that you are using. Compile your workspace, either with `catkin_make` or `catkin build`. If you are not yet familiar with ROS, the [ROS tutorials](#) are a great place to start. From this point onward, we will use common ROS-terms throughout this manual.

1.2 Source ROS Environment

At start-up, P-Rob connects to its own **roscore**. Thus, you need to set your `ROS_MASTER_URI` to the IP-address of the P-Rob (choose the Ethernet port labelled **Ethernet 2** if your P-Rob comes with more than one port).

For future improvements, we are planning to design this setup more flexible, by adding an interface to dynamically change the ROS settings, such that you can use a **roscore** on any device. For the moment, however, sourcing your ROS environment looks similar to this:

```
$ source /opt/ros/<YOUR ROS VERSION>/setup.bash
$ source <PATH TO CATKIN-WS>/devel/setup.bash
$ export ROS_MASTER_URI=http://<P-ROB DYNAMIC IP>:11311
$ export ROS_IP=<YOUR DYNAMIC IP>
```

Per se, you need to execute these lines for every open terminal window. For the sake of convenience, however, you can add them to the file `~/.bashrc` on your system, such that they are being executed automatically upon launching a new terminal. The [ROS installation instructions](#) briefly cover this as well.

2 Communicating with the ROS Module

myP is publishing data about the robot to ROS topics. Additionally, selected functions of myP can be called with ROS services.

2.1 Reading Data from Topics

You can now view the available ROS topics with the command `rostopic list`. Please note that myP only starts publishing data upon connecting. All topics are being advertised under a namespace, whose name is equal to your type of robot (e.g. `/PRob2R`).

```
/InformationPoolInput
/Pro2R/digital_inputs
/Pro2R/digital_outputs
/Pro2R/joint_currents
/Pro2R/joint_positions
/Pro2R/joint_states
/Pro2R/joint_velocities
/Pro2R/pose
/Pro2R/pose_alternative
/Pro2R/sensors
/Pro2R/transform
/node_monitor_status
/rosout
/rosout_agg
```

Unless you are operating an F&P service robot (e.g. Lio), please ignore the topics `/InformationPoolInput` and `/node_monitor_status`. The topics `/Pro2R/pose`, `/Pro2R/pose_alternative` and `/Pro2R/transform` merely use different message types to publish the same data (TCP pose of the robot). The content of the remaining topics should be clear from their names.

2.2 Calling myP Functions via Services

A useful selection of myP functions is available as ROS services. To view them, run `rosservice list`. The outer namespace is the same as for the ROS topics, e.g. `/PRob2R`. Then follows a sub-namespace for the myP core and all available add-ons. For instance, if you have purchased the add-ons `learning` and `task_manager`, the result will look similar to this (note the services for `recognize_object` and `play_task` towards the end):



```
/InformationPoolGet
/Pro2R/core/calibrate
/Pro2R/core/change_project
/Pro2R/core/close_gripper
/Pro2R/core/connect
/Pro2R/core/disconnect
/Pro2R/core/forward_kinematics
/Pro2R/core/get_grid_points
/Pro2R/core/get_status
/Pro2R/core/hold
/Pro2R/core/inverse_kinematics
/Pro2R/core/is_motor_moving
/Pro2R/core/log_message
/Pro2R/core/move_joint
/Pro2R/core/move_to_pose
/Pro2R/core/move_tool
/Pro2R/core/open_gripper
/Pro2R/core/pause
/Pro2R/core/play_path
/Pro2R/core/raise_error
/Pro2R/core/raise_warning
/Pro2R/core/read_actuator_current
/Pro2R/core/read_actuator_position
/Pro2R/core/read_digital_inputs
/Pro2R/core/read_digital_outputs
/Pro2R/core/read_gripper_angle
/Pro2R/core/read_sensor_data
/Pro2R/core/read_tcp_pose
/Pro2R/core/release
/Pro2R/core/resume
/Pro2R/core/run_simple_path
/Pro2R/core/stop
/Pro2R/core/test_script
/Pro2R/core/wait_for_motor
/Pro2R/core/write_digital_outputs
/Pro2R/learning/recognize_object
/Pro2R/task_manager/play_task
/Pro2R__myP__v1_4_0_1/get_loggers
/Pro2R__myP__v1_4_0_1/set_logger_level
/rosout/get_loggers
/rosout/set_logger_level
```



Again, unless you are operating a service robot like Lio, you can safely ignore the service `/InformationPoolGet`.

Examples

Have a look at the list below for more details on specific service commands. Since some of the ROS services are mapping to myP user script functions one-to-one, they are already covered in the [myP Script Functions Manual](#) and we are not covering them here again. The use of others, like for instance `pause` and `resume`, should be obvious from their names. The following list should cover all the remaining cases, which may require special attention.

core/connect

Before any other services become available, you need to connect to myP:

```
$ rosservice call /PRob2R/core/connect "robot_kind: 'Connect to Robot'
robot_name: 'PRob2R'"
success: True
message: ''
```

Should you not be able to establish a connection for whatever reason, you can find out about it through the success flag and the return value message. For instance, if you try to execute a script when the robot is not yet connected, the return message will look like this:

```
$ rosservice call /PRob2R/core/test_script "code: 'print(\"hello
world\")'"
type: 'main'
id: 0"
success: False
message: "Could not execute the action 'test_script'. The robot needs to
be connected."
```

Once you are connected, this command will print “hello world” in myP’s application output and return `success: True`, alongside an empty message.

core/calibrate

Next, please calibrate your robot, if you haven't already:

```
$ rosservice call /PRob2R/core/calibrate "script_id: 0
script_name: ''"
success: True
message: ''
```

Script ID 0 without script name specified simply selects the default calibration.

Note: If myP has been opened in any browser tab, a dialogue will appear in that tab asking you whether the robot is safe to move. You first need to confirm that dialogue (once) before calibration proceeds.

core/get_status

This service informs you about the status that myP is currently in:

```
$ rosservice call /PRob2R/core/get_status
status: 3
success: True
message: ''
```

In the example above, the response informed us that myP’s status was `Ready`. The meaning of the status IDs are encoded in an enum, which you can read directly from the message type of this service:

```
$ rossrv show fp_core_msgs/GetStatus
---
uint32 NONE=0
uint32 CONNECT=1
uint32 DISCONNECT=2
uint32 READY=3
uint32 STOP=4
uint32 ERROR=5
uint32 BUSY=6
uint32 RUN=7
uint32 PAUSE=8
uint32 RELEASE=9
uint32 RECORD=10
uint32 CALIBRATE=11
uint32 PROCESS=12
uint32 CHARGE=13
uint32 LISTEN=14
uint32 status
bool success
string message
```

core/change_project

Make sure that you are in the right project:

```
$ rosservice call /PRob2R/core/change_project "value: 'Default'"
success: True
message: ''
```

retrieving information through services

You can also retrieve information about the state of the robot via ROS services. For instance:

```
$ rosservice call /PRob2R/core/read_tcp_pose
pose:
  position:
    x: 0.0
    y: 0.0
    z: 1378.5
  orientation:
    phi: 0.0
    theta: 90.0
    psi: 180.0
success: True
message: ''
```

movement commands

When you use myP's ROS interface through the command line, please be careful with the default values. Since ROS autocompletes with the default values of each respective message type, the result might not be what you want. For example, if you start typing the command for `move\_joint` and then just auto-complete the message, this is what happens:

```
$ rosservice call /PRob2R/core/move_joint "actuator_ids: [0]
position: [0]
velocity: 0.0
acceleration: 0.0
block: false
relative: false"
success: False
message: "An actuator with ID ['0'] does not exist."
```

Thus you need to think about each argument and set it to the desired value. For instance, to move joint 6 to an absolute position of 90 degrees, this is probably what you want to see:

```
$ rosservice call /PRob2R/core/move_joint "actuator_ids: [6]
position: [90]
velocity: 20.0
acceleration: 100.0
block: true
relative: false"
success: True
message: ''
```

Similarly for `move_tool`, when a P-Grip is mounted in the 180-degree-configuration:

```
$ rosservice call /PRob2R/core/move_tool "x: 0.0
y: 0.0
z: 1200.0
orientation: [0, 90, 180]
velocity: 20.0
acceleration: 100.0
block: true
relative: false
frame: 'base'"
success: True
message: ''
```

core/disconnect

Once you're done, don't forget to disconnect:

```
$ rosservice call /PRob2R/core/disconnect
success: True
message: ''
```

3 Future Changes

At the time of release of myP 1.4.0, we were aware of the issues listed below. Although there exists a workaround for each of them, we need to point out that the use of these functions will still change, once the underlying issues are resolved.

- `calibrate` requires the ID of the calibration script. You can find the . Alternatively, you could send your own calibration script with the service `/PRob2R/core/test_script`.
- `release` is currently a blocking service call. `hold` either needs to be called in another console or from separate thread.
- At the moment, gripper functions are advertised under the namespace `/PRob2R/core` instead of `/PRob2R/pgrip`, although they are running inside the add-on `pgrip` (free of charge). This will change, once we revamp `pgrip`, such that its structure is consistent with the other add-ons of myP (it's a special case at the moment). Please be aware that you will then have to call those services under the new namespace.
- Essentially, the argument `robot_name` of the `connect`-service has the only effect of renaming your database, e.g. `Giggity_database.db` instead of `PRob2R_database.db`. To avoid any confusion, please make sure to always call it with `robot_name: 'PRob2R'`.

Should you encounter any issues, which are not listed above, please contact us under support@fp-robotics.com.